

Reviewer Pack — Treewidth Exponential Lower Bound on Tseitin Resolution Leng...

Entry 1c724d8f6762 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 06:14:21 UTC

Treewidth Exponential Lower Bound on Tseitin Resolution Length

Entry ID: 1c724d8f6762 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 06:14:21 UTC

1. Conjecture

Field A (mathematical branch): graph theory (treewidth) **Field B** (complexity object): resolution proof complexity

Statement:

For a graph G with treewidth $\nu(G)$, the Tseitin formula on G requires Resolution refutations of length $\geq 2^{\Omega(\nu(G))}$. $\nu(G)$ is poly-time computable and $O(1)$ for all non-expanders.

Rationale (proposer’s reasoning):

High treewidth implies complex hierarchical structure, which may necessitate exponential-length refutations in resolution due to the need to propagate constraints across multiple layers of the tree decomposition. Non-expanders (e.g., trees) have constant treewidth and admit polynomial-size refutations.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0547855953d79094

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def treewidth(G):
        if not G:
            return 0
        n = len(G)
        for u in range(n):
            neighbors = [v for v in range(n) if G[u][v]]
            if len(neighbors) == 1:
                continue
            for v in neighbors:
                G[v].remove(u)
                G[u].remove(v)
                tw = treewidth([G[i] for i in range(n) if i != u and i != v])
                G[v].append(u)
                G[u].append(v)
                if tw > 0:
                    return tw + 1
        return 0

    def tseitin_formula(G):
        n = len(G)
        literals = list(range(1, 2*n+1))
        clauses = []
        for u in range(n):
            if not G[u]:
                continue
            clause = [-literals[2*u], -literals[2*u+1]]
            clauses.append(clause)
            for v in G[u]:
                clause = [literals[2*v], literals[2*u+1]]
                clauses.append(clause)
        return clauses

    def resolution_length(clauses):
```

```

    if not clauses:
        return 0
    queue = list(clauses)
    while queue:
        c1 = queue.pop()
        for c2 in queue[:]:
            common_literals = [l for l in c1 if -l in c2]
            if not common_literals:
                continue
            new_clause = [l for l in c1 + c2 if l not in common_literals and -l not in common_literals]
            if len(new_clause) == 0:
                return float('inf')
            queue.append(new_clause)
    return len(queue)

n = random.randint(5, 40)
G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
for i in range(n):
    G[i][i] = 0

tw = treewidth(G)
formula = tseitin_formula(G)
length = resolution_length(formula)

return {
    "metric_name": "resolution_length",
    "metric_value": length,
    "instances_tested": n,
    "conjecture_holds": length >= 2**math.ceil(tw * math.log(2, 10)),
    "counterexample": "" if conjecture_holds else f"Treewidth={tw}, Length={length}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    std_length = math.sqrt(sum((r["metric_value"] - mean_length)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e16453d.py", line 92, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e16453d.py", line 74, in run_trial
    tw = treewidth(G)
         ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e16453d.py", line 32, in treewidth
    tw = treewidth([G[i] for i in range(n) if i != u and i != v])
         ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e16453d.py", line 31, in treewidth
    G[u].remove(v)
ValueError: list.remove(x): x not in list
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to ValueError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the graph node removal logic to handle missing elements, then re-run experiments with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	93418
2	propose	ollama_remote	qwen3:8b	0	0	97461
3	preregistration	ollama_remote	qwen3:8b	0	0	27376
4	novelty	ollama_remote	qwen3:8b	0	0	24115
5	novelty	ollama_remote	qwen3:8b	0	0	18801
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16333
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11059
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11679
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9981
10	judge	ollama_remote	qwen3:8b	0	0	19477

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 329701 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/1c724d8f6762.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1c724d8f6762.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1c724d8f6762.tar.gz (if generated)